

Home

Content

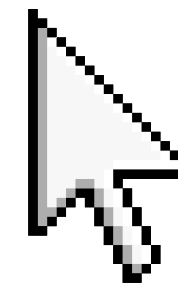
End

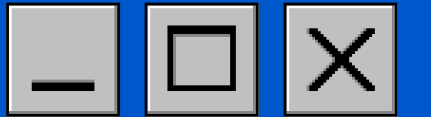
GRUPO 11

SCALA Y COBOL

PIÑEYRO - INETELLO - HEIT - WINKLER - FERRARI

Start





Home

Content

End

 **Introducción e historia de COBOL**

 **Benchmark: Scala vs COBOL**

.Búsqueda Binaria

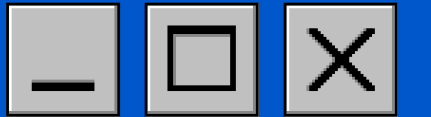
.Bubble sort

 **Conclusiones**

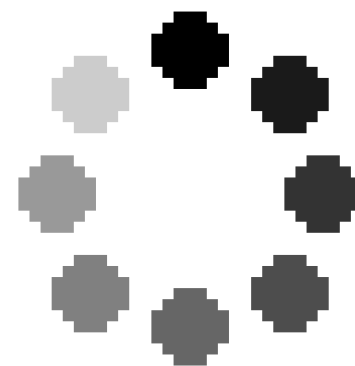


INDICE





INTRODUCCIÓN E HISTORIA DE COBOL



- ✦ El “**COmmon Business-Oriented Language**” (COBOL) es un lenguaje de programación compilado de alto nivel, estandarizado como lenguaje informático en 1968.
- ✦ Desarrollado específicamente para satisfacer las necesidades de procesamiento de datos empresariales.
- ✦ Es un derivado en parte de FLOW-MATIC, un lenguaje creado por la pionera de la informática, la **Dra. Grace Hopper**.
- ✦ Fue de los primeros lenguajes de programación normalizados por el Instituto Nacional Estadounidense de Estándares (ANSI) y la Organización Internacional de Normalización (ISO).
- ✦ Actualmente es menos popular que otros lenguajes más modernos, pero sigue siendo una parte funcional y crítica de la infraestructura tecnológica mundial.



[Home](#)[Content](#)[End](#)

Uno de los grandes puntos fuertes de COBOL es su fuerte soporte para cálculos decimales de punto fijo de gran precisión.

La sintaxis de COBOL es **autodocumentada** y casi **autoexplicativa**, con énfasis en la verbosidad y la legibilidad.

Estructura

Identification division

Environment division

Data division

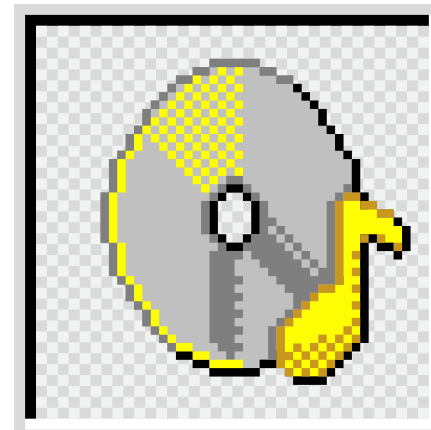
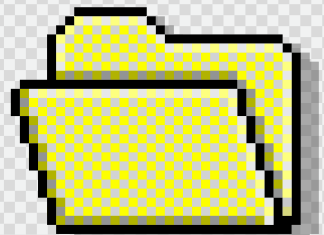
Procedure division

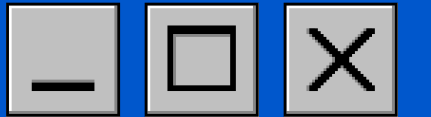
Ventajas



-  Gran estabilidad y fiabilidad
-  Permite escalar aplicaciones
-  Gran capacidad de procesamiento
-  Comunicación con aplicaciones web

SOBRE COBOL



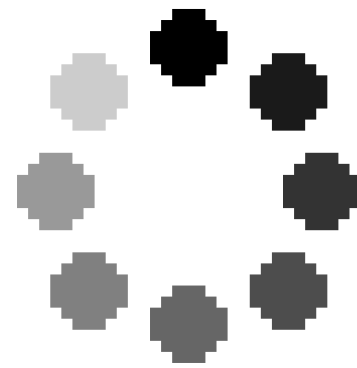


Home

Content

End

BENCHMARK SCALA VS COBOL



COBOL

BUSQUEDA BINARIA

Scala

```
01 TABLA-VALORES.
    05 VALOR OCCURS 100 TIMES PIC 9(4).

01 IND-INICIO      PIC 9(3) VALUE 1.
01 IND-FIN         PIC 9(3) VALUE 100.
01 IND-MEDIO       PIC 9(3).
01 VALOR-BUSCADO   PIC 9(4) VALUE 500.
01 HALLADO         PIC X(10) VALUE "NOENCONTR".

01 I               PIC 9(3).

PROCEDURE DIVISION.

    PERFORM VARYING I FROM 1 BY 1
        UNTIL I > 100

        COMPUTE VALOR(I) = I * 10

    END-PERFORM

    PERFORM UNTIL IND-INICIO > IND-FIN
        OR HALLADO = "ENCONTRADO"

        COMPUTE IND-MEDIO =
            (IND-INICIO + IND-FIN) / 2

        IF VALOR(IND-MEDIO) = VALOR-BUSCADO

            MOVE "ENCONTRADO" TO HALLADO

        ELSE

            IF VALOR(IND-MEDIO) < VALOR-BUSCADO

                COMPUTE IND-INICIO = IND-MEDIO + 1
            ELSE
                COMPUTE IND-FIN = IND-MEDIO - 1

            END-IF

        END-IF
    END-PERFORM

    IF HALLADO = "ENCONTRADO"
        DISPLAY "ELEMENTO ENCONTRADO EN LA POSICION: "
            IND-MEDIO
    ELSE
        DISPLAY "ELEMENTO NO ENCONTRADO EN LA TABLA"
    END-IF
    STOP RUN.
```

```
import scala.annotation.tailrec

object BusquedaBinariaFuncional extends App {

    def busquedaBinaria(arreglo: Array[Int], elementoBuscado: Int): Option[Int] = {

        @tailrec
        def buscar(inicio: Int, fin: Int): Option[Int] = {
            if (inicio > fin) {
                None // Caso base: Elemento no encontrado
            } else {
                val medio = inicio + (fin - inicio) / 2

                arreglo(medio) match {
                    case valor if valor == elementoBuscado => Some(medio)
                    case valor if valor < elementoBuscado => buscar(medio + 1, fin)
                    case _ => buscar(inicio, medio - 1)
                }
            }
        }

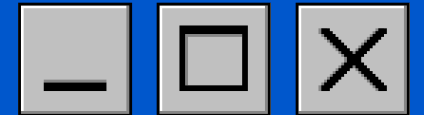
        buscar(0, arreglo.length - 1)
    }

    // Ejemplo de uso:
    val datos = Array(2, 5, 8, 12, 16, 23, 38, 56, 72, 91)
    val objetivo = 23

    val inicio = System.nanoTime()
    val resultado = busquedaBinaria(datos, objetivo)
    val fin = System.nanoTime()

    resultado match {
        case Some(indice) =>
            println(s"Elemento encontrado en el índice: $indice")
        case None =>
            println("Elemento no encontrado")
    }

    println(s"Tiempo: ${((fin - inicio) / 1000000.0)} ms")
}
```

[Home](#)[Content](#)[End](#)

ANÁLISIS RENDIMIENTO

Compilado en: GnuCOBOL



Cada vector utilizado contiene $n = 10000$ (la complejidad del algoritmo es $O(\log n)$ ya que en cada paso reduce el espacio de búsqueda a la mitad. Por lo tanto el máx, de comparaciones es 13.



COBOL:
TIEMPO: 1 ms



SCALA:
TIEMPO: 1.58 ms



COBOL

BUBBLE SORT

Scala

```
01 CANTIDAD      PIC 9(5) VALUE 1000.
01 PASADA        PIC 9(5).
01 INDICE        PIC 9(5).
01 TEMPORAL      PIC 9(5).

01 VECTOR.
   05 NUMERO OCCURS 1000 TIMES PIC 9(5).

PROCEDURE DIVISION.

   PERFORM VARYING INDICE FROM 1 BY 1
      UNTIL INDICE > CANTIDAD
      COMPUTE NUMERO(INDICE) = CANTIDAD - INDICE + 1
   END-PERFORM

   PERFORM VARYING PASADA FROM 1 BY 1
      UNTIL PASADA >= CANTIDAD

      PERFORM VARYING INDICE FROM 1 BY 1
         UNTIL INDICE > CANTIDAD - PASADA

         IF NUMERO(INDICE) > NUMERO(INDICE + 1)
            MOVE NUMERO(INDICE) TO TEMPORAL
            MOVE NUMERO(INDICE + 1) TO NUMERO(INDICE)
            MOVE TEMPORAL TO NUMERO(INDICE + 1)
         END-IF

      END-PERFORM

   END-PERFORM

   DISPLAY "ORDENAMIENTO FINALIZADO".

   STOP RUN.
```

```
object BBS {

  def bubbleSort(vector: Array[Int]): Unit = {
    val cantidad = vector.length

    for (pasada <- 0 until cantidad - 1) {
      for (indice <- 0 until cantidad - 1 - pasada) {
        if (vector(indice) > vector(indice + 1)) {
          val temporal = vector(indice)
          vector(indice) = vector(indice + 1)
          vector(indice + 1) = temporal
        }
      }
    }
  }

  def main(args: Array[String]): Unit = {

    val cantidad = 1000

    val vector = Array.ofDim[Int](cantidad)

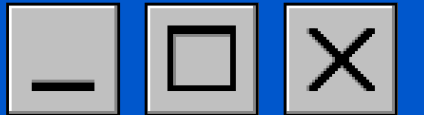
    for (indice <- 0 until cantidad) {
      vector(indice) = cantidad - indice
    }

    val inicio = System.nanoTime()

    bubbleSort(vector)

    val fin = System.nanoTime()

    println(s"Tiempo: ${((fin - inicio) / 1000000.0)} ms")
  }
}
```

[Home](#)[Content](#)[End](#)

ANÁLISIS RENDIMIENTO

Compilado en: GnuCOBOL



Cada vector utilizado contiene $n = 1000$ (la complejidad del algoritmo es aproximadamente $O(n^2)$ porque compara repetidamente cada elemento con los demás).

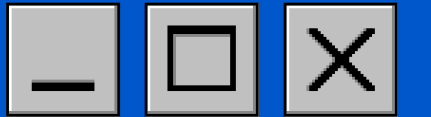


COBOL:
TIEMPO: 5 ms

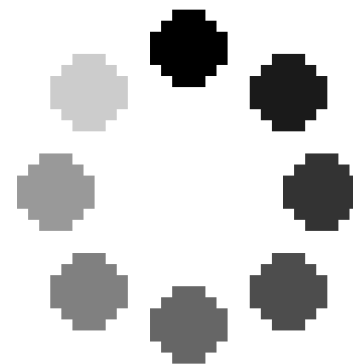


SCALA:
TIEMPO: 17 ms





CONCLUSIONES



RESEÑA



- Modelo de ejecución

COBOL se compila como un programa ejecutable, mientras que Scala se compila a bytecode y se ejecuta sobre la Java Virtual Machine (JVM).



- Gestión de memoria

En los programas analizados, COBOL utiliza estructuras de tamaño fijo definidas previamente, mientras que Scala delega la administración de memoria a la JVM mediante el Garbage Collector.



- Optimización

Scala puede beneficiarse de las optimizaciones realizadas por el compilador JIT (Just-In-Time) durante la ejecución. En COBOL, el rendimiento está determinado principalmente por el código generado por el compilador.



- Rendimiento

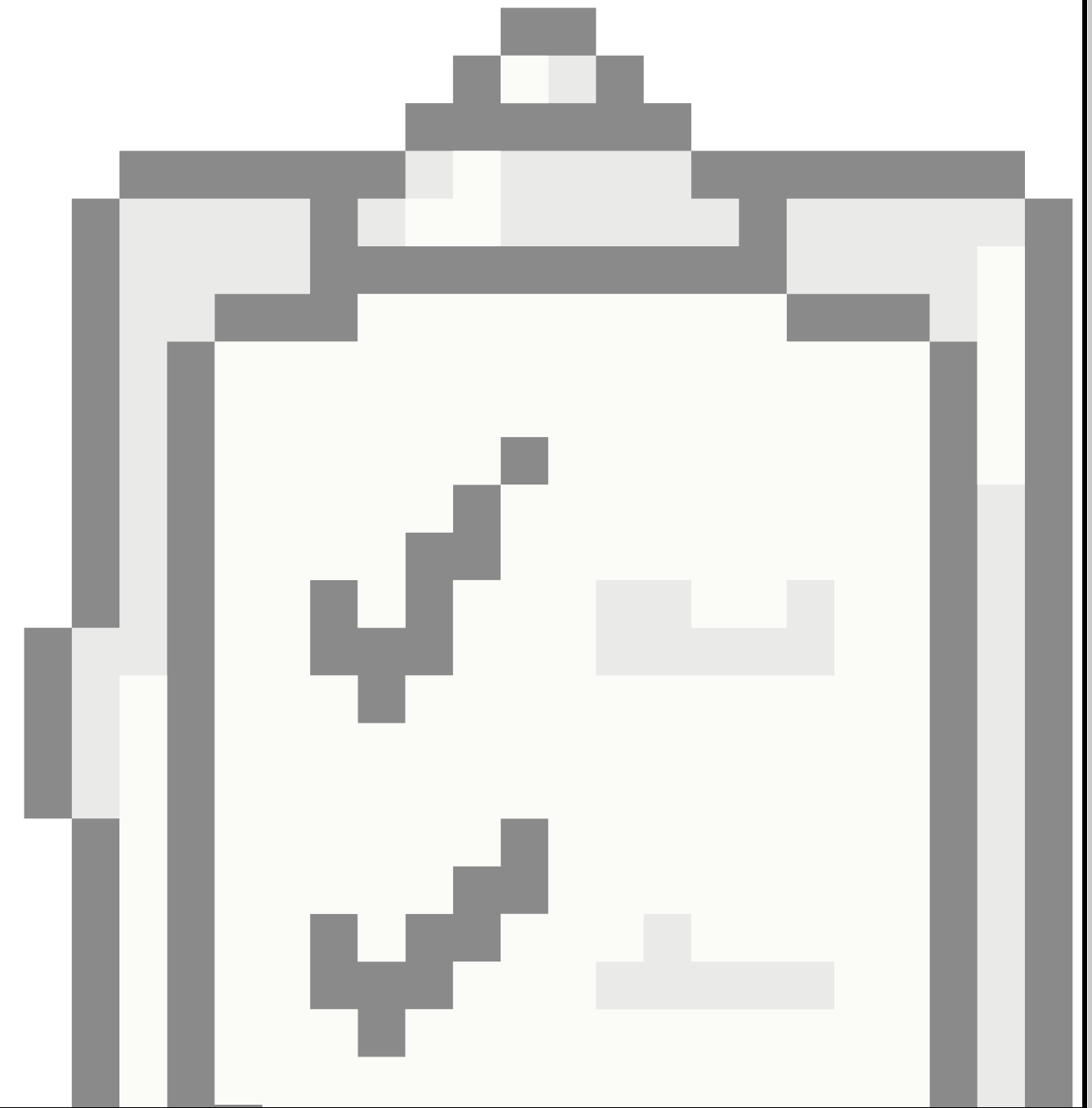
En algoritmos con pocas operaciones, como la búsqueda binaria, ambos lenguajes presentan tiempos muy similares. En algoritmos más costosos, como Bubble Sort, las diferencias de rendimiento dependen del compilador y del entorno de ejecución utilizado.

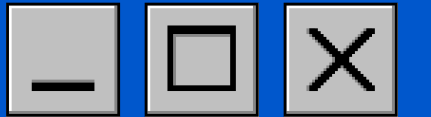


EN CONCLUSIÓN



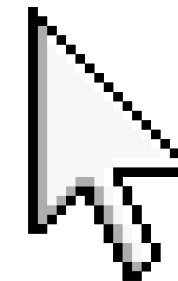
-  **COBOL** continúa siendo un lenguaje muy utilizado en sistemas empresariales gracias a su estabilidad, fiabilidad y capacidad para procesar grandes volúmenes de datos.
-  Cada lenguaje está pensado para necesidades distintas, por lo que la elección depende del problema que se quiera resolver.
-  El rendimiento no depende únicamente del lenguaje, sino también del algoritmo utilizado y del entorno de ejecución.
(y que compilador se use)





GRACIAS
TOTALES

Exit



PREGUNTAS PARA LA CLASE :)

¿Qué condición debe cumplir un arreglo para poder aplicar búsqueda binaria?

¿Para qué tipo de aplicaciones fue creado COBOL?

¿Sobre qué plataforma se ejecuta Scala?

¿Qué significa COBOL?

